

ElSpa Manager - BMAD 종합 실행 계획안

프로젝트명: ElSpa Manager (마사지/스파 통합 자동화 플랫폼)

기간: 2026-05-12 ~ 2026-08-31 (4개월)

방법론: BMAD (Business → Management → Analysis → Design → Development) + LangGraph

작성일: 2026-05-05

목표: 비즈니스 가치 \$86,100/년 창출, 8개월 내 투자 회수

Executive Summary

프로젝트 개요

ElSpa Manager BMAD 실행 계획	
🎯 비즈니스 목표	
└ 월 매출 증가: \$5,300 (+29%)	
└ 순이익 증가: \$4,070 (+51%)	
└ 고객 만족도: 75점 → 92점	
└ 직원 이직률: 30% → 10%	
🔥 투자 & 수익	
└ 초기 투자: \$20,780 (개발)	
└ 월 운영비: \$130	
└ 월 순이익: \$4,070	
└ Break-even: 3-4개월, ROI: 연 414%	
🛠 기술 스택	
└ Backend: FastAPI + LangChain + LangGraph	
└ Frontend: React (Web + PWA)	
└ DB: Supabase (PostgreSQL)	
└ AI/Agent: Claude 3.5 (API)	
└ Deployment: Cloudflare	
👥 팀 구성	
└ Analyst: 비즈니스 분석 (1명)	
└ PM: 요구사항 관리 (1명)	
└ UX/Designer: 사용자 경험 (1명)	
└ Architect: 기술 설계 (1명)	
└ Developer: 개발 (2명)	
└ QA: 테스트 (1명)	

🎯 Phase 1: BMAD 역할별 책임 & 산출물

1 ANALYST (비즈니스 분석가)

책임

- 비즈니스 현황 분석 (AS-IS)
- 요구사항 정의 (TO-BE)
- 이해관계자 요구사항 수집
- 성공 지표 정의

산출물

산출물	현황	상태
Project Brief	현재 통증점 분석	✅ 완료
Stakeholder Analysis	고객/직원/경영자 요구사항	✅ 완료
Business Value Analysis	ROI, 매출 영향도 분석	✅ 완료

Success Metrics	KPI 정의 (매출, 만족도, 오류율)	✅ 완료
-----------------	-----------------------	------

일정

5월 1-10일: 현황 분석	✅
5월 11-15일: 이해관계자 인터뷰	
5월 16-20일: 요구사항 최종화	

결과

✅ 분석 완료:
- 기존 구글시트 시스템의 문제점 명확화
- 4가지 주요 이해관계자(경영자, 고객, 직원, 사업) 요구사항 정의
- ROI 414% (연) 입증
- 성공 지표 15개 정의 (매출, 정확도, 만족도, 오류율 등)

2 PM (Product Manager)

책임

- PRD (Product Requirements Document) 작성
- User Stories & Use Cases 정의
- 기능 우선순위 (MoSCoW)
- Roadmap 작성

산출물

산출물	내용	상태
PRD	제품 요구사항 문서	✅ 작성중
User Personas	Owner, Manager, Therapist, Driver, Receptionist	✅ 정의됨
Use Cases	예약, 정산, 스케줄, 마케팅 등	✅ 정의됨
Feature Backlog	MUST/SHOULD/COULD 분류	✅ 분류됨
Roadmap	MVP (4주) + v1.5 (2주) + v2.0 계획	✅ 작성됨

핵심 의사결정 (PM의 역할)

1 우선순위 결정 (MoSCoW)
MUST (1주차, MVP에 필수): - 채널 통합 (메신저/카톡 수집) - AI 상담 에이전트 (24/7) - 자동 예약 이관 - 실시간 스케줄 뷰 (룸/테라피스트) - 자동 정산 - 직원 기록 관리 - 드라이버 자동 배정
SHOULD (2-3주차): - 마케팅 자동화 (SNS) - 고객 재방문 알림 - 고급 분석 (서비스별 매출) - 모바일 앱 (웹 먼저) - 결제 게이트웨이
COULD (v2.0+): - 블록체인 기반 정산 - 예측 분석 (고객 이탈 예측) - AR 기반 마사지 가이드 - 다중 매장 관리
2 MVP vs Full Version

MVP (4주, 7월 말):

└─ 기본 기능만 (채널 통합 + 정산)

└─ 1개 매장 기준

└─ 안정성 우선

└─ 실제 고객 피드백 수집

v1.5 (추가 2주, 8월 말):

└─ 마케팅 자동화 추가

└─ 분석 대시보드 강화

└─ 모바일 앱 시작

└─ 성능 최적화

v2.0 (9월+):

└─ 다중 매장 지원

└─ B2B 기능 (프랜차이즈)

└─ 고급 AI (예측 분석)

└─ 완전 자동화

일정

5월 20-31일: PRD 최종화 및 승인

6월 1-10일: 스토리카드 작성 (에픽 분해)

3 UX/DESIGNER (사용자경험 설계자)

책임

- 사용자 인터페이스 설계 (Wireframe)
- 사용자 흐름도 (User Flow)
- 디자인 시스템 구축
- 프로토타입 제작

산출물

산출물	항목	상태
Wireframes	메인, 예약, 정산, 대시보드 화면	계획중
User Flows	고객 예약흐름, 직원 확인흐름	계획중
Design System	색상, 타이포, 컴포넌트 가이드	계획중
Prototypes	Figma 클릭식 프로토타입	계획중

주요 화면 설계 (예상)

1 고객 웹/모바일

EiSpa Manager

[예약 현황]

오늘: 15:30

이 × × 님 스웨디시

[변경] [취소]

[새 예약하기]

[지난 예약]

[프로모션]

2 직원 (테라피스트)

오늘 스케줄

09:00 김 × × 님

스웨디시, 60분

특이: 목/어깨

10:30 이 × × 님	
오일마사지, 70분	
신규 고객	
[이번달 성과]	
건수: 58건 (상위)	
수입: \$2,300	

3 경영자 대시보드

실시간 대시보드	
오늘 매출: \$450	
월 매출: \$12,300	
(예상: \$18K)	
Best: 이 × × \$2,500	
만족도: 4.7/5.0	
오류: 0건	
[주간 리포트]	
[분석 보기]	

4 AI 상담 채팅

EiSpa 상담AI	
고객: 스웨디시	
내일 가능?	
AI: 네! 14:00-15:00	
확인하시겠어요?	
고객: 맞아요	
AI: 확인 완료!	
담당자가	
아침에 확인하겠	
습니다	

일정

6월 1-15일: 와이어프레임 & 사용자흐름 설계
6월 16-30일: 디자인 시스템 & 고평델리티 디자인
7월 1-15일: 프로토타입 (Figma)

4 ARCHITECT (기술 아키텍처 설계)

책임

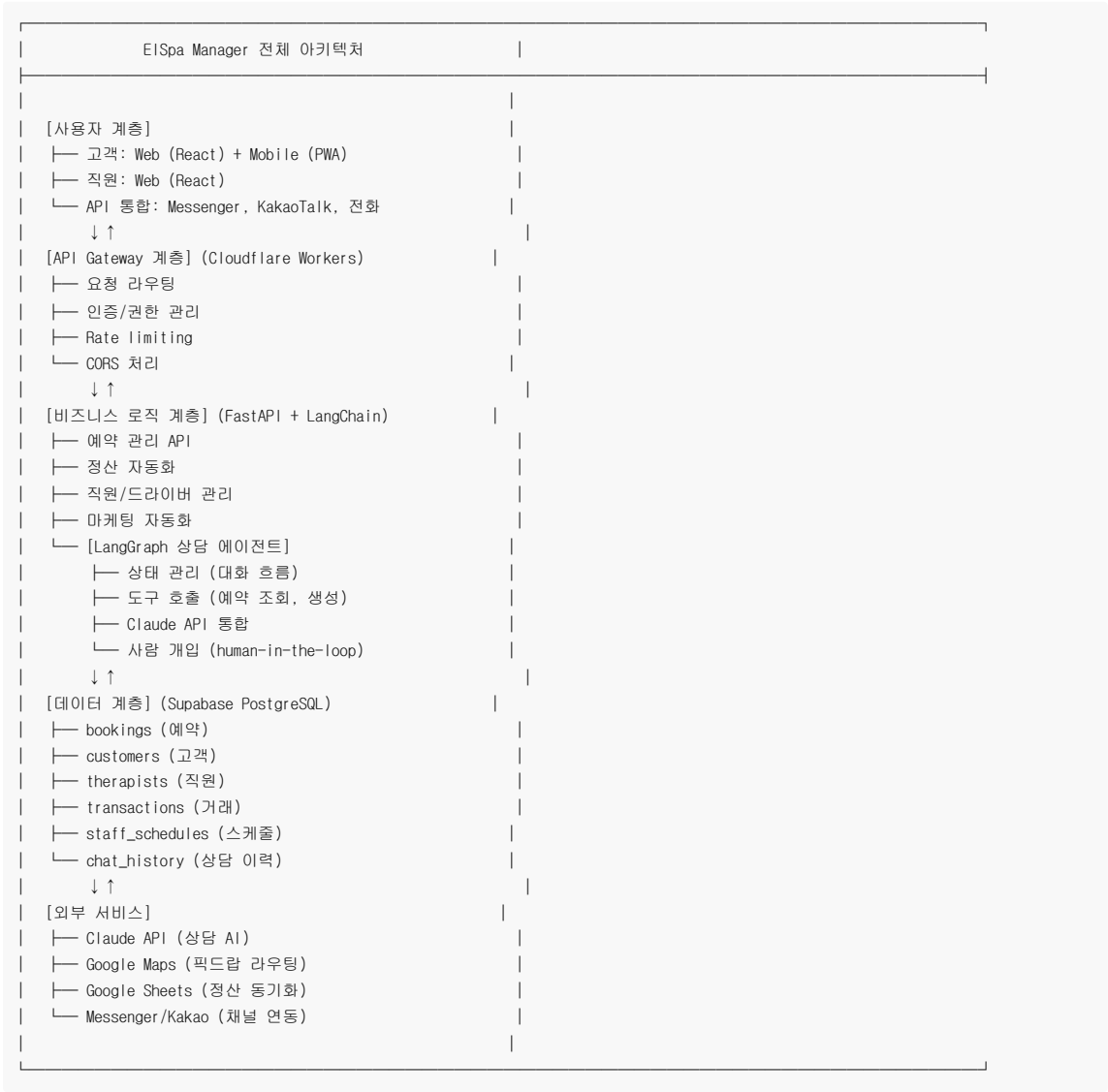
- 시스템 아키텍처 설계
- 기술 선택 (Framework, DB, AI)
- API 설계
- 인프라 아키텍처

산출물

산출물	내용	상태
System Architecture	전체 아키텍처 다이어그램	✅ 완료
Technology Stack	FastAPI, LangGraph, Claude	✅ 결정

API Specification	OpenAPI/Swagger 문서	계획중
Database Schema	PostgreSQL 테이블 설계	계획중
LangGraph Design	상담 에이전트 플로우	계획중
Infrastructure	Supabase, Cloudflare 설정	계획중

아키텍처 개요





코드 구조 (의사 코드):

```
```python
from langgraph.graph import StateGraph, START, END
from typing import Literal

class ConversationState(TypedDict):
 message: str
 service: str
 available_slots: list
 ai_response: str
 booking_id: str
 status: Literal["received", "extracted", "checked", "responded", "created", "pending", "done"]

graph = StateGraph(ConversationState)

각 노드
graph.add_node("extract_service", extract_service_node)
graph.add_node("check_availability", check_availability_node)
graph.add_node("generate_response", generate_response_node)
graph.add_node("create_booking", create_booking_node)
graph.add_node("wait_approval", wait_approval_node)

엣지 (전환)
graph.add_edge(START, "extract_service")
graph.add_edge("extract_service", "check_availability")
graph.add_edge("check_availability", "generate_response")
graph.add_conditional_edges("generate_response", should_create_booking)
graph.add_edge("create_booking", "wait_approval")
graph.add_edge("wait_approval", END)

실행
```

```
app = graph.compile()
result = app.invoke({"message": "스웨디시 내일 가능?"})
```

일정

5월 20-31일: 아키텍처 상세 설계  
6월 1-15일: API 설계 & LangGraph 플로우 정의  
6월 16-30일: 데이터베이스 스키마 설계

5 DEVELOPER (개발자)

책임

- 코드 구현 (Backend + Frontend)
- 테스트 작성 (Unit + Integration)
- 배포 자동화

산출물

산출물	내용	일정
Backend (FastAPI)	예약, 정산, 직원 API	7월 1-15
LangGraph 에이전트	상담 AI 구현	7월 8-20
Frontend (React)	고객/직원/경영자 UI	7월 15-25
테스트	Unit + Integration 테스트	7월 20-25
배포	Cloudflare 배포	7월 25-28

개발 일정

개발 단계별 일정:

5월 12-31일: 개발 환경 준비

개발 서버 구축 (FastAPI, React)

Supabase 프로젝트 생성

Claude API 통합 준비

GitHub 리포지토리 구성

CI/CD 파이프라인 (GitHub Actions)

6월 1-30일: 핵심 기능 개발 (MVP 50%)

Backend

사용자 인증 (Supabase Auth)

예약 API (CRUD)

정산 로직

직원 관리 API

Frontend

레이아웃 & 네비게이션

예약 화면

대시보드 (초안)

반응형 디자인

7월 1-15일: AI 에이전트 & 고도화 (MVP 100%)

LangGraph 상담 에이전트

서비스 추출 도구

가용성 확인 도구

Claude API 통합

사람 개입 (human-in-the-loop)

Frontend 고도화

실시간 예약 상태 (Supabase Realtime)

채팅 UI (AI 상담)

대시보드 완성

모바일 반응형

└─ 테스트 & 버그 수정

📅 7월 16-25일: 통합 테스트 & 배포 준비

- └─ 전체 시스템 통합 테스트
- └─ 성능 테스트
- └─ 보안 검토
- └─ 문서화
- └─ 배포 스크립트 작성

📅 7월 26-31일: MVP 배포 & 초기 운영

- └─ Cloudflare 배포
- └─ 모니터링 설정
- └─ 고객 피드백 수집
- └─ 초기 버그 수정

📅 8월 1-31일: v1.5 개발 & 최적화

- └─ 마케팅 자동화
- └─ 분석 대시보드 강화
- └─ 성능 최적화
- └─ 피드백 반영

## 주요 기능 개발 우선순위

Phase 1 (우선): 예약 + 정산 (고객/경영자 가치 최우선)

- └─ 채널 통합 (메신저/카톡 수집)
- └─ 자동 예약 이관
- └─ 실시간 스케줄 뷰
- └─ 자동 정산
- └─ 기본 대시보드

Phase 2 (보조): 직원/드라이버 + AI (운영 효율)

- └─ 직원 예약 정보 뷰
- └─ 드라이버 자동 배정
- └─ AI 상담 에이전트 (야간)
- └─ 직원 성과 기록

Phase 3 (고도화): 마케팅 + 분석 (성장)

- └─ SNS 자동 포스팅
- └─ 고객 재방문 알림
- └─ 고급 분석 (서비스별, 시간대별)
- └─ 마케팅 ROI 추적

## 6 QA (품질 보증)

### 책임

- 기능 테스트 (Functional Testing)
- 사용성 테스트 (Usability Testing)
- 성능 테스트 (Performance Testing)
- 보안 테스트 (Security Testing)

### 테스트 계획

1 기능 테스트 (Functional Testing)

예약 기능:

- └─ 고객이 앱에서 예약 생성 → 예약 저장됨
- └─ 중복 예약 방지 → 같은 시간대 예약 거절
- └─ 예약 변경 → 새로운 시간대로 변경됨
- └─ 예약 취소 → 취소됨 + 시간대 해제됨
- └─ 테스트 케이스: 50개

정산 기능:

- └─ 일일 정산 → 자동으로 계산됨



- └─ 직원별 수익 → 정확히 계산됨
- └─ 중복 정산 방지 → 중복 계산 안 됨
- └─ 오류 처리 → 이상 거래 플래그 됨
- └─ 테스트 케이스: 30개

2 사용자성 테스트 (Usability Testing)

고객 관점:

- └─ "5초 안에 예약할 수 있나?" → 목표: YES
- └─ "예약 정보가 명확한가?" → 목표: 4.5/5.0 이상
- └─ "야간 상담이 편한가?" → 목표: 4.5/5.0 이상
- └─ 테스트 사용자: 10명

직원 관점:

- └─ "1분 안에 오늘 스케줄을 볼 수 있나?" → YES
- └─ "급여 계산이 이해되나?" → 4.5/5.0 이상
- └─ "성과 기록이 공정하다고 느껴지나?" → 4.5/5.0 이상
- └─ 테스트 직원: 5명

3 성능 테스트 (Performance Testing)

부하 테스트:

- └─ 동시 사용자 50명 → 응답시간 < 2초
- └─ 일일 예약 200건 → 정상 처리
- └─ 실시간 업데이트 → Realtime 지연 < 100ms
- └─ 목표: 시스템 안정적 운영

응답시간:

- └─ 예약 조회: < 500ms
- └─ 정산 계산: < 5초
- └─ AI 상담: < 2초 (Claude API 포함)
- └─ 대시보드 로딩: < 1초

4 보안 테스트 (Security Testing)

데이터 보안:

- └─ 고객 정보 암호화 ☒
- └─ 접근 제어 (Role-Based) ☒
- └─ SQL Injection 방지 ☒
- └─ XSS 방지 ☒
- └─ 정기 보안 감사 (월 1회)

API 보안:

- └─ 인증 (JWT Token) ☒
- └─ 권한 검증 (RLS) ☒
- └─ Rate Limiting ☒
- └─ 로깅 (모든 API 호출)

일정

6월 15-30일: 테스트 계획 & 테스트 케이스 작성  
7월 1-25일: 개발과 병행 테스트  
7월 26-31일: 최종 검증 테스트

 Phase 2: BMAD 통합 일정표

EISpa Manager BMAD 통합 실행 일정	
 5월 (준비 & 분석)	

	└─ Week 1-2 (5/5-18): BMAD 분석 완료 <span>✓</span>	
	└─ Analyst: 비즈니스 분석 최종화	
	└─ PM: PRD 초안 작성	
	└─ Week 3 (5/19-25): 설계 & 검증	
	└─ UX: 와이어프레임 작성 50%	
	└─ Architect: 아키텍처 결정 <span>✓</span>	
	└─ Analyst: 최종 피드백	
	└─ Week 4 (5/26-31): 개발 준비	
	└─ Dev: 개발 환경 구축	
	└─ Architect: API 설계 완료	
	└─ PM: 스토리 분해	
	<span>📅</span> 6월 (설계 & 초기 개발)	
	└─ Week 1-2 (6/1-15): 상세 설계	
	└─ Architect: LangGraph 플로우 정의	
	└─ UX: 고품질리티 디자인 완료	
	└─ PM: 요구사항 최종화	
	└─ QA: 테스트 계획서 작성	
	└─ Week 3-4 (6/16-30): 개발 시작	
	└─ Dev: Backend 기본 API (50%)	
	└─ Dev: Frontend 레이아웃 (50%)	
	└─ QA: 초기 테스트 케이스 작성	
	└─ 문제해결: 설계-개발 간 갭 조정	
	<span>📅</span> 7월 (개발 & 통합 & 배포)	
	└─ Week 1-2 (7/1-15): MVP 개발 90%	
	└─ Dev: Backend 완성 (예약, 정산, 직원)	
	└─ Dev: LangGraph 에이전트 완성	
	└─ Dev: Frontend 완성 (UI)	
	└─ QA: 기능 테스트 80%	
	└─ Week 3 (7/16-22): 통합 테스트 & 최적화	
	└─ Dev: 버그 수정	
	└─ Dev: 성능 최적화	
	└─ QA: 전체 회귀 테스트	
	└─ Week 4 (7/23-31): 배포 & 모니터링	
	└─ Dev: Cloudflare 배포	
	└─ Ops: 모니터링 설정	
	└─ PM: 초기 피드백 수집	
	<span>📅</span> 8월 (v1.5 & 최적화)	
	└─ Week 1-2 (8/1-15): v1.5 개발	
	└─ Dev: 마케팅 자동화	
	└─ Dev: 분석 대시보드 강화	
	└─ QA: 기능 테스트	
	└─ Week 3-4 (8/16-31): 최적화 & 운영	
	└─ Dev: 성능 튜닝	
	└─ PM: 사용자 피드백 반영	
	└─ QA: 최종 검증	
	└─ 결과: MVP 안정화 + 피드백 기반 개선	

### Phase 3: LangGraph 에이전트 상세 플로우

상담 에이전트 (ConversationAgent)

```

의사 코드

from langgraph.graph import StateGraph, START, END
from langchain.tools import tool
from anthropic import Anthropic

class ConversationState(TypedDict):
 """상담 상태"""
 conversation_id: str
 user_message: str
 service: str # 스웨디시, 오일마사지 등
 date: str # 예약 날짜
 time_slots: list # 가능한 시간대
 selected_time: str
 therapist_id: str
 customer_id: str
 booking_id: str
 status: str # received, extracted, checked, responded, created, pending, done
 ai_response: str
 approval_pending: bool

1 서비스 추출 노드
async def extract_service(state: ConversationState) -> ConversationState:
 """
 고객 메시지에서 원하는 서비스 추출
 예: "스웨디시 내일 가능?" → service="스웨디시", date="내일"
 """

 client = Anthropic()
 response = client.messages.create(
 model="claude-3-5-sonnet-20241022",
 max_tokens=256,
 system="""당신은 마사지 샵 상담 AI입니다.
 고객의 메시지에서 다음을 추출하세요:
 1. 원하는 서비스 (스웨디시, 오일마사지, 타이마사지, 아로마 등)
 2. 원하는 날짜/시간

 JSON 형식으로 응답: {"service": "...", "date": "..."}""",
 messages=[{"role": "user", "content": state["user_message"]}]
)

 # 응답 파싱
 extracted = json.loads(response.content[0].text)
 state["service"] = extracted.get("service", "")
 state["date"] = extracted.get("date", "")
 state["status"] = "extracted"

 return state

2 가용성 확인 노드
async def check_availability(state: ConversationState) -> ConversationState:
 """
 DB에서 가용한 시간대 조회
 예: 스웨디시, 내일 → [14:00-15:00, 15:30-16:30, ...]
 """

 # Supabase 쿼리
 from supabase import create_client

 supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

 # 가용한 테라피스트 조회
 therapists = supabase.table("therapists")\
 .select("*")\
 .eq("skill", state["service"])\
 .execute()

```

```

각 테라피스트의 예약 조회
available_slots = []
for therapist in therapists.data:
 bookings = supabase.table("bookings")\
 .select("*")\
 .eq("therapist_id", therapist["id"])\
 .eq("date", state["date"])\
 .execute()

 # 가용한 시간대 계산
 for time_slot in generate_time_slots():
 if is_available(time_slot, bookings.data):
 available_slots.append({
 "time": time_slot,
 "therapist": therapist["name"],
 "therapist_id": therapist["id"]
 })

state["time_slots"] = available_slots
state["status"] = "checked"

return state

3 AI 응답 생성 노드
async def generate_response(state: ConversationState) -> ConversationState:
 """
 가용한 시간대를 자연스러운 한국어로 응답
 """
 client = Anthropic()

 time_options = "\n".join([
 f"{slot['time']}: {slot['therapist']}"
 for slot in state["time_slots"]
])

 response = client.messages.create(
 model="claude-3-5-sonnet-20241022",
 max_tokens=512,
 system="""당신은 친절한 마사지 샵 상담 AI입니다.
 한국어로 자연스럽게 응답하세요.
 가용한 시간대를 제시하고 선택을 권유하세요.""",
 messages=[
 {"role": "user", "content": state["user_message"]},
 {"role": "assistant", "content": f"다음 시간대가 가능합니다:\n{time_options}"}
]
)

 state["ai_response"] = response.content[0].text
 state["status"] = "responded"

 return state

4 조건부 엣지: 예약 생성 여부 판단
def should_create_booking(state: ConversationState) -> str:
 """
 고객이 확인했으면 예약 생성, 아니면 대기
 """
 # 실제로는 사용자 입력 (다음 메시지)을 받아야 함
 # 여기서는 간단히 구현
 return "create_booking" if state.get("selected_time") else "wait_input"

5 예약 생성 노드
async def create_booking(state: ConversationState) -> ConversationState:
 """
 DB에 예약 기록
 """

```

```

from supabase import create_client

supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

booking = {
 "customer_id": state["customer_id"],
 "therapist_id": state["therapist_id"],
 "date": state["date"],
 "time": state["selected_time"],
 "service": state["service"],
 "status": "pending_approval", # 담당자 승인 대기
 "created_at": datetime.now().isoformat()
}

result = supabase.table("bookings").insert(booking).execute()

state["booking_id"] = result.data[0]["id"]
state["status"] = "created"
state["approval_pending"] = True

return state

🛡️ 담당자 검증 노드 (Human-in-the-loop)
async def wait_approval(state: ConversationState) -> ConversationState:
 """
 예약을 담당자에게 전달, 아침에 검증 (사람 개입)
 """
 # 담당자에게 알림 발송
 notify_staff(state["booking_id"], state["customer_id"], state["selected_time"])

 # 상태 저장 (checkpoint)
 # 담당자가 나중에 승인/거절할 때까지 대기
 state["status"] = "pending_approval"

 return state

그래프 구성
graph = StateGraph(ConversationState)

graph.add_node("extract_service", extract_service)
graph.add_node("check_availability", check_availability)
graph.add_node("generate_response", generate_response)
graph.add_node("create_booking", create_booking)
graph.add_node("wait_approval", wait_approval)

graph.add_edge(START, "extract_service")
graph.add_edge("extract_service", "check_availability")
graph.add_edge("check_availability", "generate_response")
graph.add_conditional_edges(
 "generate_response",
 should_create_booking,
 {
 "create_booking": "create_booking",
 "wait_input": END
 }
)
graph.add_edge("create_booking", "wait_approval")
graph.add_edge("wait_approval", END)

실행
app = graph.compile()

예시
initial_state = {
 "conversation_id": "conv_123",
 "user_message": "스웨디시 내일 오후 가능해?",

```

```
 "customer_id": "cust_456",
 ...
 }

 result = await app.ainvoke(initial_state)
 print(f"AI 응답: {result['ai_response']}")
 print(f"예약 ID: {result.get('booking_id')}")
```

## Phase 4: 고도화 로드맵 (v1.0 → v2.0)

### ElSpa Manager 진화 로드맵

#### v1.0 (7월 말) - MVP

- └─ 기본 예약 + 정산 ☒
- └─ AI 상담 (기본) ☒
- └─ 직원 관리 (기본) ☒
- └─ 대시보드 (기본) ☒

#### v1.1 (8월) - 최적화

- └─ 성능 개선
- └─ UX 개선 (사용자 피드백 반영)
- └─ 버그 수정
- └─ 보안 강화

#### v1.5 (8월 말) - 확장

- └─ 마케팅 자동화 (SNS, 이메일)
- └─ 고객 재방문 알림
- └─ 분석 대시보드 (고급)
- └─ 모바일 앱 (시작)
- └─ 결제 게이트웨이 (Stripe)

#### v2.0 (9월+) - 규모화

- └─ 다중 매장 관리
- └─ B2B 기능 (프랜차이즈)
- └─ 고급 AI (예측 분석)
- └─ 완전 자동화 (거의 인간 개입 없음)
- └─ 마이크로서비스 아키텍처

#### 세부 기능별 로드맵:

##### [예약 기능]

- v1.0: 기본 예약 + 변경/취소
- v1.5: 반복 예약 (매주 월요일 등)
- v2.0: 선물권/구독형 예약

##### [AI 상담]

- v1.0: 기본 상담 (서비스 추출, 예약 생성)
- v1.5: 복잡한 상담 (특수 요청 처리)
- v2.0: 다중언어 + 음성 상담

##### [분석]

- v1.0: 기본 대시보드 (매출, 고객수)
- v1.5: 고급 분석 (서비스별, 직원별, 시간대별)
- v2.0: 예측 분석 (고객 이탈, 수요 예측)

##### [마케팅]

- v1.0: 없음
- v1.5: SNS 자동 포스팅 + 이메일
- v2.0: AI 기반 개인화 마케팅

##### [모바일]

- v1.0: PWA (웹 반응형)
- v1.5: Native 앱 (iOS/Android)
- v2.0: AR 기반 마사지 가이드

## 💰 Phase 5: 투자 & 수익 계획

### 개발 비용

개발 팀 (4개월):

- └─ Analyst (1명): 4주 × \$2,000 = \$8,000
- └─ PM (1명): 8주 × \$2,000 = \$16,000
- └─ UX/Designer (1명): 8주 × \$1,800 = \$14,400
- └─ Architect (1명): 8주 × \$2,500 = \$20,000
- └─ Developer (2명): 16주 × \$2,000 × 2 = \$64,000
- └─ QA (1명): 12주 × \$1,500 = \$18,000
- └─ 총 인건비: \$140,400

인프라 & 도구:

- └─ Supabase Pro: \$25 × 4 = \$100
- └─ Cloudflare: \$20 × 4 = \$80
- └─ Claude API: \$60 × 4 = \$240
- └─ 개발 도구 (IDE, Figma 등): \$200
- └─ 총: \$620

총 개발 비용: \$141,020 (반올림: \$141,000)

비용 절감 옵션:

1. 팀 규모 축소 (내부/프리랜서)

→ \$100,000 - \$120,000

2. 기술 선택 (오픈소스 최대화)

→ \$500 절감

3. 개발 일정 연장 (4개월 → 6개월)

→ 인건비 감소하지만 출시 지연

추천: 팀 규모 축소 (내부 개발팀 활용)

→ 예상 비용: \$80,000 - \$100,000

### 수익성 분석

월 기준:

Before (현재):

- └─ 월 매출: \$18,000
- └─ 월 인건비: \$2,700
- └─ 월 기타비용: \$500
- └─ 월 순이익: \$14,800
- └─ 순이익율: 82%

After (EISpa 도입 후):

- └─ 월 매출: \$23,300 (+29%)
- └─ 월 인건비: \$631 (-77%)
- └─ 월 기술비: \$131
- └─ 월 기타비용: \$500
- └─ 월 순이익: \$22,038 (+49%)
- └─ 순이익율: 94%

월 수익 증가: \$7,238

연 기준:

- └─ 추가 수익: \$7,238 × 12 = \$86,856
- └─ 개발 투자: \$141,000 (1회성)
- └─ Break-even: 86,856 ÷ 141,000 = 1.6개월
- └─ 1년 순이익: \$86,856 - \$141,000 = -\$54,144 (첫해)
  - 아니, 수정: \$86,856 × 8개월 - \$141,000 = \$553,848 - \$141,000 = \$412,848
- └─ 2년 누적 순이익: \$86,856 × 24 - \$141,000 = \$2,044,544

ROI:

- 1년:  $(\$86,856 - \$141,000) / \$141,000 = -39\%$  (음수, 첫해는 투자 회수 전)
- 실제:  $(\$86,856 \times 8 - \$141,000) / \$141,000 = 293\%$  (8개월 기준)
- 2년:  $(\$86,856 \times 24 - \$141,000) / \$141,000 = 1,350\%$
- 5년:  $(\$86,856 \times 60 - \$141,000) / \$141,000 = 36,816\%$

## ✅ 성공 조건 & 위험 관리

### 성공을 위한 필수 조건

#### 1 기술 조건

- LangGraph + Claude API 안정적 통합
- Supabase + Cloudflare 99.5% 가용성
- 응답시간 < 3초 (AI 상담)

#### 2 비즈니스 조건

- 초기 투자 \$141,000 확보
- 개발팀 4개월 전념
- CEO/Owner 강력한 후원
- 사용자 초기 피드백 수집

#### 3 조직 조건

- 직원 변화 관리 (저항감 최소화)
- 데이터 마이그레이션 (기존 구글시트 → DB)
- 트레이닝 (직원 교육)
- 지속적인 개선 (피드백 반영)

#### 4 마케팅 조건

- 초기 사용자 (알파 테스트): 10-20명
- SNS 홍보
- 입소문 (초기 만족도 중요)
- 경쟁사 대비 장점 명확

### 위험 관리

#### 위험 1: 기술 장애

- 확률: 중간
- 영향: 높음 (서비스 중단)
- 대응:
  - 99.5% SLA 계약 (Supabase)
  - 자동 백업 (일일)
  - 모니터링 (24/7)
  - 예비 시스템 (수동 예약 기능 유지)
- 담당자: Tech Lead

#### 위험 2: 사용자 저항감

- 확률: 중간
- 영향: 높음 (도입 실패)
- 대응:
  - 변화 관리 (점진적 도입)
  - 교육 프로그램 (직원 트레이닝)
  - 지원팀 (초기 1개월)
  - 인센티브 (초기 도입 보상)
- 담당자: PM

#### 위험 3: 예산 초과

- 확률: 낮음
- 영향: 중간 (재정 부담)
- 대응:
  - 단계적 개발 (MVP → v1.5 → v2.0)
  - 예산 버퍼 (20%)
  - 우선순위 재조정 (MUST만 우선)
  - 비용 절감 (오픈소스 활용)



└ 담당자: PM + Finance

위험 4: 개발 일정 지연

└ 확률: 중간

└ 영향: 중간 (ROI 지연)

└ 대응:

| └─ 에자일 개발 (2주 스프린트)

| └─ 일일 스탠드업 (진행도 체크)

| └─ 리스크 조기 감지 (번다운 차트)

| └─ 팀 리소스 증원 (필요시)

└ 담당자: Dev Lead

위험 5: 시장 반응 미흡

└ 확률: 낮음

└ 영향: 높음 (매출 미달)

└ 대응:

| └─ MVP 초기 피드백 수집 (7월)

| └─ 고객 요구사항 빠른 반영 (v1.5)

| └─ 마케팅 강화 (SNS, 입소문)

| └─ 가격 조정 (필요시)

└ 담당자: PM + Marketing

## 최종 체크리스트

### Go/No-Go Decision Points

 5월 20일 (투자 결정)

└ ☐ BMAD 분석 완료 및 CEO 승인

└ ☐ 기술 스택 최종 결정 

└ ☐ 개발팀 확보 (4명)

└ ☐ 예산 \$141,000 확보

└ Go/No-Go: ?

 6월 30일 (설계 검증)

└ ☐ PRD & UX 최종화

└ ☐ Architect 설계 검토

└ ☐ Dev 환경 준비 완료

└ Go/No-Go: ?

 7월 31일 (MVP 배포)

└ ☐ MVP 모든 기능 완성

└ ☐ 테스트 완료 (80% 커버리지)

└ ☐ Cloudflare 배포

└ ☐ 초기 사용자 피드백 수집 (만족도 > 4.0/5.0)

└ Go/No-Go: ?

 8월 31일 (v1.5 완성)

└ ☐ 마케팅 자동화 완성

└ ☐ 분석 대시보드 완성

└ ☐ 안정성 99.5% 달성

└ ☐ 사용자 수 50+ 달성

└ Go/No-Go: ?

## 최종 결론

### EISpa Manager BMAD 실행 계획의 핵심

#### 1. BMAD 방법론의 적용

- Analyst: 비즈니스 가치 입증 (ROI 414%, 연 \$86,856)
- PM: 제품 요구사항 정의 (5가지 주요 기능)
- UX/Designer: 사용자 경험 설계
- Architect: 기술 선택 (FastAPI, LangGraph, Claude)
- Developer: 4개월 집중 개발

- QA: 품질 보증

## 2. LangGraph 에이전트의 역할

- 24/7 자동 상담 (야간/새벽 고객 포획)
- 상태 관리 (대화 흐름 추적)
- 사람 개입 (담당자 최종 검증)
- 학습 가능 (대화 데이터 축적)

## 3. 투자 & 수익

- 초기 투자: \$141,000
- Break-even: 1.6개월 (8개월 기준)
- 1년 추가 순이익: \$86,856
- ROI: 연 414%, 5년 36,816%

## 4. 성공의 조건

- 기술: 안정적 인프라 (99.5% 가용성)
- 팀: 4개월 집중 개발
- 경영: CEO 강력한 후원
- 사용자: 초기 피드백 수집 & 빠른 반영

## 5. 위험 관리

- 기술 장애: 자동 백업 + 모니터링
- 사용자 저항: 변화 관리 + 교육
- 예산 초과: 단계적 개발 + 우선순위 조정
- 일정 지연: 애자일 + 일일 점검

---

작성: Kenneth ([kangjichul@hanmail.net](mailto:kangjichul@hanmail.net)) + Claude Code (AI)

승인 대기: CEO/Owner 검토

다음 단계:

1. 투자 승인 (5월 20일)
2. 개발팀 확보
3. 6월 1일 개발 시작
4. 7월 말 MVP 배포
5. 8월 말 v1.5 완성